

This project has retired. For details please refer to its Attic page.

Introduction

Apache Giraph is an iterative graph processing framework, built on top of **Apache Hadoop**.

The input to a Giraph computation is a graph composed of vertices and directed edges, see Figure 1. For example vertices can represent people, and edges friend requests. Each vertex stores a *value*, so does each edge. The input, thus, not only determines the graph topology, but also the initial values of vertices and edges.

As an example, consider a computation that finds the distance from a predetermined *source person* s to any person in the social graph. In this computation, the value of an *edge* E is a floating point number denoting distance between adjacent people. The value of a *vertex* V is also a floating point number, representing an upper bound on the distance along a shortest path from the predetermined vertex s to v . The initial value of the predetermined source vertex s is 0, and the initial value for any other vertex is infinity.

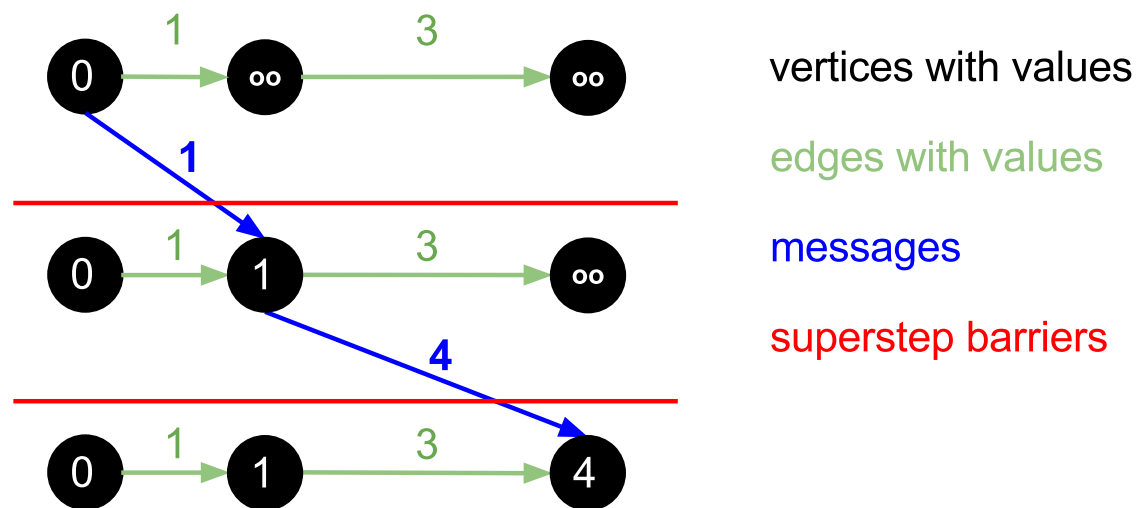


Figure 1: An illustration of an execution of a single source shortest paths algorithm in Giraph. The input is a chain graph with three vertices (black) and two edges (green). The values of edges are 1 and 3 respectively. The algorithm computes distances from the leftmost vertex. The initial values of vertices are 0, ∞ and ∞ (top row). Distance upper bounds are sent as messages (blue),

Project Information ▾ User Docs ▾ Developer Docs ▾

Computation proceeds as a sequence of iterations, called *supersteps* in [BSP](#) . Initially, every vertex is *active*. In each superstep each active vertex invokes the *Compute method* provided by the user. The method implements the graph algorithm that will be executed on the input graph. Intuitively, you should think like a vertex when designing a Giraph algorithm, it is vertex oriented. A graph oriented approach is discussed in [GIRAPH-818](#) .

The Compute method:

- receives messages sent to the vertex in the previous superstep,
- computes using the messages, and the vertex and outgoing edge values, which may result in modifications to the values, and
- may send messages to other vertices.

The Compute method does not have direct access to the values of other vertices and their outgoing edges. Inter-vertex communication occurs by sending messages.

In our single-source shortest paths example, a Compute method will: (1) find the minimum value arriving on any message, (2) if that value is less than the current value of the vertex, then (3) the minimum will be adopted as the vertex value, and (4) the value plus the edge value will be sent along every outgoing edge. See a simplified code in [Figure 2](#) and a complete Java implementation [here](#) .

```
public void compute(Iterable<DoubleWritable> messages) {
    double minDist = Double.MAX_VALUE;
    for (DoubleWritable message : messages) {
        minDist = Math.min(minDist, message.get());
    }
    if (minDist < getValue().get()) {
        setValue(new DoubleWritable(minDist));
        for (Edge<LongWritable, FloatWritable> edge : getEdges()) {
            double distance = minDist + edge.getValue().get();
            sendMessage(edge.getTargetVertexId(), new DoubleWritable(distance));
        }
    }
    voteToHalt();
}
```

Figure 2: The Compute method for a single source shortest paths computation. Each vertex in the graph executes this method at each superstep. The method computes a minimum distance arriving on messages and can send distances along each edge.

Project Information ▾

User Docs ▾

Developer Docs ▾



current superstep.

The graph can be mutated during computation by adding or removing vertices or edges. Our example shortest paths algorithm does not mutate the graph.

Values are retained across barriers. That is, the value of any vertex or edge at the beginning of a superstep is equal to the corresponding value at the end of the previous superstep, when graph topology is not mutated. For example, when a vertex has set the distance upper bound to D , then at the beginning of the next superstep the distance upper bound will still be equal D . Of course the vertex can modify the value of the vertex and of the outgoing edges during any superstep.

Any vertex can stop computing after any superstep. The vertex simply declares that it does not want to be active anymore. However, any incoming message will make the vertex active again.

The computation halts after the vertices have voted to halt and there are no messages in flight. Each vertex outputs some local information, which usually amounts to the final vertex value.

Copyright © 2011-2020 [The Apache Software Foundation](#). All Rights Reserved.

